University Examination Answers

Module 3: Django Admin Interfaces and Model Forms

Q1. Activating and Configuring the Django Admin Interface *(10 Marks)*

Django's automatic admin interface works by reading metadata in your model to provide a powerful and production-ready interface that site administrators can start using immediately.

Steps to Activate the Admin Interface

1. **Add admin metadata to your models.**

Not all models can or should be editable by admin users, so you need to mark models that should have an admin interface. You do this by adding an inner Admin class to your model:

```
class Book(models.Model):
    title = models.CharField(maxlength=100)
    authors = models.ManyToManyField(Author)
    publisher = models.ForeignKey(Publisher)
    publication_date = models.DateField()
    def __str__(self):
        return self.title
    class Admin:
        pass    # Admin declaration — flags this model for admin interface
```

The Admin declaration flags the class as having an admin interface. The pass keyword signifies an empty Admin class, using all defaults.

2. **Install the admin application.**

Add "django.contrib.admin" to your INSTALLED_APPS setting in settings.py. Also ensure that the following are uncommented:

- "django.contrib.sessions"
- "django.contrib.auth"
- "django.contrib.contenttypes"

Also uncomment all lines in the MIDDLEWARE_CLASSES setting tuple.

3. **Run database migrations.**

Run python manage.py syncdb. This installs the extra database tables the admin interface uses.

> **Note:** *When you first run syncdb with 'django.contrib.auth' in INSTALLED_APPS, you will be asked to create a superuser. Without a superuser account, you cannot log in to the admin interface.*

4. **Add the URL pattern in urls.py.**

```
from django.conf.urls.defaults import *
urlpatterns = patterns('',
    (r'^admin/', include('django.contrib.admin.urls')),
)
```

5. **Start the development server.**

Run python manage.py runserver and visit http://127.0.0.1:8000/admin/ in your browser. You will be greeted by the login screen, where you enter the superuser credentials.

What You Can Do After Logging In

- Each model with an Admin declaration appears on the main admin index page.
- Links to 'add' and 'change' objects appear — these lead to change lists and edit forms.
- Edit forms handle different field types with appropriate widgets (calendar for dates, select for foreign keys, etc.).
- Input validation is automatically handled — leaving a required field blank shows errors.
- A History button logs every change made through the admin interface.
- Deletions show a cascade confirmation page listing all related objects that will be deleted.

Q2. Handling and Processing Forms in Django *(10 Marks)*

Django's forms framework is designed to make form construction, validation, and processing as easy as possible. The 'perfect form' should display a form, validate input, redisplay with errors, and process valid data.

Step 1 — Define the Form Class

Forms in Django are created declaratively using a Python class. By convention, forms are placed in a forms.py file within your application directory:

```
from django import newforms as forms

TOPIC_CHOICES = (
    ('general', 'General enquiry'),
    ('bug', 'Bug report'),
    ('suggestion', 'Suggestion'),
)

class ContactForm(forms.Form):
    topic = forms.ChoiceField(choices=TOPIC_CHOICES)
    message = forms.CharField(widget=forms.Textarea())
    sender = forms.EmailField(required=False)
```

A Django form is a subclass of django.newforms.Form, just as a Django model is a subclass of django.db.models.Model.

Step 2 — Hook the Form into a View

The view handles both GET (display empty form) and POST (process submitted form) requests:

```
from django.http import HttpResponseRedirect
from django.shortcuts import render_to_response
from django.core.mail import send_mail
from forms import ContactForm

def contact(request):
    if request.method == 'POST':
        form = ContactForm(request.POST)  # Bound form
        if form.is_valid():
            topic   = form.clean_data['topic']
            message = form.clean_data['message']
            sender  = form.clean_data.get('sender', 'noreply@example.com')
            send_mail(
                'Feedback: %s' % topic,
                message, sender,
```

```
                ['administrator@example.com']
            )
            return HttpResponseRedirect('/contact/thanks/')  # Redirect after POST
    else:
        form = ContactForm()  # Unbound form
    return render_to_response('contact.html', {'form': form})
```

Step 3 — Create the Template

```
<html><body>
<h1>Contact us</h1>
<form action="." method="POST">
    <table>{{ form.as_table }}</table>
    <p><input type="submit" value="Submit"></p>
</form>
</body></html>
```

Key Concepts

- Bound vs Unbound: A bound form is constructed with data (request.POST) and can be validated. An unbound form has no data and simply knows how to display itself.
- is_valid(): Tells whether a form is bound to valid data. Triggers field validators and custom clean methods.
- form.clean_data: A dictionary of validated, type-converted field values. Always use this instead of request.POST directly.
- Redirect after POST: After a successful POST, redirect the user to avoid duplicate submissions on page refresh.
- form.as_table() / as_ul() / as_p(): Convenience methods to render the form as HTML table rows, list items, or paragraphs respectively.

Q3. URLconf Tricks — List and Explanation *(10 Marks)*

URLconfs are just Python code, which gives developers the flexibility to use several powerful tricks. Chapter 8 of The Django Book covers the following URLconf tricks:

1. Streamlining Function Imports

Instead of importing each view function individually at the top of urls.py, you can pass a string containing the module name and function name:

```
urlpatterns = patterns('mysite.views',
    (r'^now/$', 'current_datetime'),
    (r'^now/plus/(\d{1,2})hours/$', 'hours_ahead'),
)
```

The common prefix 'mysite.views' is factored out as the first argument to patterns(), avoiding repetition. Django automatically imports the appropriate view function when first needed.

- Why: Reduces repetitive import statements; results in more manageable URLconfs.

2. Using Multiple View Prefixes

When views come from different modules (no common prefix), you can combine multiple patterns() objects:

```
urlpatterns = patterns('mysite.views',
    (r'^/?$', 'archive_index'),
)
```

```
urlpatterns += patterns('weblog.views',
    (r'^tag/(\w+)/$', 'tag'),
)
```

- Why: Allows each group of URLs to use its own prefix, keeping the URLconf clean and DRY.

3. Special-Casing URLs in Debug Mode

URLconf patterns can be constructed dynamically at runtime. You can conditionally add debug-only URLs:

```
from django.conf import settings
urlpatterns = patterns('', (r'^$', 'mysite.views.homepage'))
if settings.DEBUG:
    urlpatterns += patterns('', (r'^debuginfo$', 'mysite.views.debug'))
```

- Why: Restricts sensitive debug views to development environments only.

4. Using Named Groups

In Python regex, named groups use the syntax (?P<name>pattern). Captured values are passed as keyword arguments instead of positional arguments:

```
urlpatterns = patterns('',
    (r'^articles/(?P<year>\d{4})/(?P<month>\d{2})/$', views.month_archive),
)
# Results in: month_archive(request, year='2006', month='03')
```

- Why: Makes URLconfs more explicit, less prone to argument-order bugs, and more readable.

5. Passing Extra Options to View Functions

Each URLconf pattern can include a third item — a dictionary of keyword arguments:

```
urlpatterns = patterns('',
    (r'^foo/$', views.foobar_view, {'template_name': 'template1.html'}),
    (r'^bar/$', views.foobar_view, {'template_name': 'template2.html'}),
)
```

- Why: Allows a single reusable view function to handle multiple similar URLs without code duplication.

6. Using Default View Arguments

Multiple URL patterns can point to the same view, with the view defining default parameter values:

```
urlpatterns = patterns('',
    (r'^blog/$', views.page),
    (r'^blog/page(?P<num>\d+)/$', views.page),
)
def page(request, num='1'):  # default num is '1'
    ...
```

- Why: Keeps view logic DRY when multiple URLs share the same underlying logic.

7. Special-Casing Views

Because URLconfs are processed top-to-bottom, a specific URL pattern can be placed before a general one to override it (short-circuit logic):

```
urlpatterns = patterns('',
    ('^auth/user/add/$', 'django.contrib.admin.views.auth.user_add_stage'),
    ('^([^/]+)/([^/]+)/add/$', 'django.contrib.admin.views.main.add_stage'),
)
```

- Why: Handles edge cases cleanly without putting URL logic inside view functions.

8. Capturing Text in URLs

All captured URL values are passed as plain Python strings, regardless of the regex pattern. Always convert types explicitly (e.g., int(year)) to avoid TypeErrors in view code.

- Why: Prevents subtle bugs when passing captured values to functions that require specific types (e.g., datetime).

9. Determining What URLconf Searches Against

Django matches URLconf patterns against the requested URL path — not including domain, GET/POST parameters, or the leading slash.

- Why: Simplifies patterns by removing the need to account for host names or query strings.

Q4. ModelForm for Student Project *(10 Marks)*

A ModelForm in Django automatically generates a form based on a model's fields, eliminating the need to redefine fields in both the model and the form. Below is the complete implementation:

models.py — The Project Model

```
from django.db import models

LANGUAGE_CHOICES = (
    ('python', 'Python'),
    ('java', 'Java'),
    ('cpp', 'C++'),
    ('javascript', 'JavaScript'),
    ('php', 'PHP'),
)

class Project(models.Model):
    topic       = models.CharField(maxlength=200,
                    verbose_name='Project Topic')
    languages   = models.CharField(maxlength=100,
                    choices=LANGUAGE_CHOICES,
                    verbose_name='Languages Used')
    duration    = models.IntegerField(
                    verbose_name='Duration (in weeks)')

    def __str__(self):
        return self.topic

    class Admin:
        pass
```

forms.py — The ModelForm

```
from django import newforms as forms
from models import Project

class ProjectForm(forms.ModelForm):
    class Meta:
        model = Project  # Bind to the Project model
        # All fields: topic, languages, duration
```

views.py — The View to Handle the Form

```
from django.http import HttpResponseRedirect
```

```
from django.shortcuts import render_to_response
from forms import ProjectForm

def add_project(request):
    if request.method == 'POST':
        form = ProjectForm(request.POST)
        if form.is_valid():
            form.save()  # Saves the data directly to the database
            return HttpResponseRedirect('/projects/success/')
    else:
        form = ProjectForm()  # Empty unbound form
    return render_to_response('add_project.html', {'form': form})
```

urls.py

```
from django.conf.urls.defaults import *
urlpatterns = patterns('',
    (r'^projects/add/$', 'mysite.views.add_project'),
)
```

add_project.html — Template

```
<html><body>
<h2>Add Student Project</h2>
<form action="." method="POST">
    <table>{{ form.as_table }}</table>
    <p><input type="submit" value="Save Project"></p>
</form>
</body></html>
```

> **Note:** *ModelForms automatically generate form fields from model fields, validate input, and provide a save() method that writes data directly to the database.*

Q5. Customizing the Admin Interface

Django allows customization of the admin interface through the Admin inner class of a model, as well as through Django's template system.

Customizing the Change List Display (Admin class options)

The Admin class declaration controls how models appear and behave in the admin interface. Example for a Book model:

```
class Book(models.Model):
    title = models.CharField(maxlength=100)
    authors = models.ManyToManyField(Author)
    publisher = models.ForeignKey(Publisher)
    publication_date = models.DateField()

    class Admin:
        list_display  = ('title', 'publisher', 'publication_date')
        list_filter   = ('publisher', 'publication_date')
        ordering      = ('-publication_date',)
        search_fields = ('title',)
```

- list_display: Controls which columns appear in the change list table. By default, only a single column with the object's string representation is shown.

- list_filter: Creates a filtering sidebar on the right side of the list. Works best with foreign keys, dates, Booleans, and fields with a choices attribute.
- ordering: Controls the order of objects in the admin. A minus sign prefix reverses the order. In the example above, most recent publications appear first.
- search_fields: Creates a text search box. Allows searching by the specified fields.

Customizing the Look and Feel

The Django admin site is powered by Django's own template system. To customize it, copy the relevant stock admin template into a directory listed in TEMPLATE_DIRS.

To change the 'Django administration' header: copy django/contrib/admin/templates/admin/base_site.html to your custom template directory and edit it to replace the generic text with your site's name.

Any default admin template can be overridden using the same approach — copy it from the default directory to your custom directory and make changes.

Customizing the Admin Index Page

The admin index template is admin/index.html. It uses a template tag called {% get_admin_app_list as app_list %} to retrieve installed apps. You can hard-code links to object-specific admin pages or run python manage.py adminindex <app> to get a chunk of template code as a starting point.

Q6. Creating a Feedback Form and Processing the Submission

Django's forms framework enables the construction of robust, validated forms with minimal code. Below is the complete feedback form implementation from the textbook.

Step 1 — Define the Form (forms.py)

```
from django import newforms as forms

TOPIC_CHOICES = (
    ('general', 'General enquiry'),
    ('bug', 'Bug report'),
    ('suggestion', 'Suggestion'),
)

class ContactForm(forms.Form):
    topic   = forms.ChoiceField(choices=TOPIC_CHOICES)
    message = forms.CharField(widget=forms.Textarea())
    sender  = forms.EmailField(required=False)
```

Step 2 — Write the View with Processing Logic (views.py)

```
from django.http import HttpResponseRedirect
from django.shortcuts import render_to_response
from django.core.mail import send_mail
from forms import ContactForm

def contact(request):
    if request.method == 'POST':
        form = ContactForm(request.POST)
        if form.is_valid():
            topic   = form.clean_data['topic']
            message = form.clean_data['message']
```

```
            sender  = form.clean_data.get('sender', 'noreply@example.com')
            send_mail(
                'Feedback from your site, topic: %s' % topic,
                message,
                sender,
                ['administrator@example.com']
            )
            return HttpResponseRedirect('/contact/thanks/')
    else:
        form = ContactForm()
    return render_to_response('contact.html', {'form': form})
```

Step 3 — Template (contact.html)

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN">
<html lang="en"><head><title>Contact us</title></head>
<body>
<h1>Contact us</h1>
<form action="." method="POST">
    <table>{{ form.as_table }}</table>
    <p><input type="submit" value="Submit"></p>
</form>
</body></html>
```

How Processing Works

- On GET request: an unbound ContactForm() is created and rendered as an empty form.
- On POST request: a bound form ContactForm(request.POST) is created with the submitted data.
- form.is_valid(): runs all validators. If valid, form.clean_data holds type-converted field values.
- send_mail(): sends the feedback email using the four required arguments — subject, body, from address, list of recipients.
- HttpResponseRedirect: redirects to a success page after processing, avoiding duplicate submissions on refresh.

Q7. Benefits of Using Django Admin Interfaces *(5 Marks)*

- Automatic Generation: The admin interface is built automatically from model metadata with just a few lines of code (an inner Admin class). There is no need to build a custom admin UI from scratch.
- Production-Ready Immediately: The interface is powerful and production-ready from the moment it is activated — administrators can begin adding, editing, and deleting data right away.
- Data Entry for Non-Technical Users: The admin is designed for nontechnical users. At Django's origin newspaper, reporters could enter data while programmers focused on the public-facing interface.
- Input Validation: The admin interface automatically validates all input. Leaving required fields blank, or entering invalid data (e.g., wrong date format), raises errors without any extra code.
- Change History: Every change made through the admin interface is logged. The History button on any object lets administrators review all modifications.
- Users, Groups, and Permissions: Built-in user management system. Permissions can be assigned per model (add, change, delete). Users can be grouped for bulk permission assignment.
- Cascade Delete Confirmation: When deleting objects, the admin shows a confirmation page listing all related objects that will also be deleted, preventing costly mistakes.

- Inspecting Data Models: Developers use the admin to quickly inspect new models, enter dummy data, and identify data-modelling mistakes early.

Q8. Django Template System Basics — How It Differs

A Django template is a string of text that is intended to separate the presentation of a document from its data. A template defines placeholders and various bits of basic logic (template tags) that regulate how the document should be displayed.

Core Components

- Variables: Written as {{ variable }}. They are replaced with the value of the named variable from the context. Variable names are case-sensitive and can use dot notation to access attributes.
- Template Tags: Written as {% tag %}. They tell the template system to 'do something' — control flow (if, for), load libraries, or define blocks. Many require a closing tag ({% endif %}, {% endfor %}).
- Filters: Applied using the pipe character as {{ variable|filter }}. They transform the display of a variable without modifying the underlying data. Filters can be chained.

Using the Template System

The two-step process: (1) Create a Template object by providing raw template code as a string. (2) Call the render() method of the Template object with a Context (a dictionary of variables).

```
from django.template import Template, Context
t = Template('My name is {{ name }}.')
c = Context({'name': 'Stephane'})
print t.render(c)   # Output: 'My name is Stephane.'
```

> **Note:** *If template code includes any syntax errors, the Template() constructor raises a TemplateSyntaxError immediately.*

How Django Templates Differ from Other Systems

- Intentional limitations: Django templates deliberately cannot execute arbitrary Python code or call functions with arguments. This enforces strict separation of logic (views) and presentation (templates).
- Non-Pythonic syntax: Unlike Jinja2 or Mako (which allow full Python expressions), Django templates use a restricted tag and filter language — designers without Python knowledge can work with them safely.
- Security focus: Filters like escape are built-in to prevent XSS. The system fails silently (renders empty string) rather than crashing when variables are missing.
- Reusability: The same Template object can be rendered multiple times with different Context objects efficiently, since compilation to internal form happens only once.

Q9. Custom Validation in Django Forms — Example

Custom validation in Django forms is implemented by adding clean_<fieldname>() methods to the form class. These are called after the default field validator, allowing additional business logic to be applied.

Example — Validating Message Length

The textbook illustrates a validation rule requiring messages to be at least four words:

```
class ContactForm(forms.Form):
    topic   = forms.ChoiceField(choices=TOPIC_CHOICES)
    message = forms.CharField(widget=forms.Textarea())
    sender  = forms.EmailField(required=False)

    def clean_message(self):
        message   = self.clean_data.get('message', '')
        num_words  = len(message.split())
        if num_words < 4:
            raise forms.ValidationError('Not enough words!')
        return message   # IMPORTANT: always return the (possibly modified) value
```

How It Works

- The method is named clean_<fieldname> — Django automatically finds and calls it during validation.
- The field data has already been partially processed by the default validator, so it is pulled from self.clean_data.
- If validation fails, raise forms.ValidationError() with a descriptive message. This message is displayed to the user as an item in the error list.
- If validation passes, the method must explicitly return the value. Forgetting the return statement causes None to be returned and the original value is lost.
- The clean_message method can also transform the value (e.g., strip whitespace, normalise case) before returning it.

Q10. Model Forms in Django — Importance and Differences *(10 Marks)*

A ModelForm is a special type of Django form that is automatically generated from a model's field definitions. It eliminates the need to redefine fields in both the model and the form.

Why Model Forms Are Important

- Avoids duplication: Without ModelForms, developers must define each field twice — once in the model and again in the form class. ModelForms derive the form fields automatically from the model.
- Automatic validation: Field constraints defined in the model (e.g., maxlength, blank=False) are automatically enforced in the form.
- Direct database saving: ModelForms provide a save() method that writes validated form data directly to the database in one line.
- Consistency: Any change to the model (e.g., adding a new field) is automatically reflected in the form without manual updates.

Comparison: Regular Form vs. ModelForm

Feature	Regular Form (forms.Form)
Field definition	Fields must be explicitly defined in the form class
Model binding	No direct model connection
Saving data	Developer must manually extract data and save to model
Validation	Only form-level validators; model constraints not auto-applied
Use case	Custom forms not tied to a specific model (e.g., contact form)

Feature	ModelForm (forms.ModelForm)
Field definition	Fields derived automatically from model fields
Model binding	Bound to a specific model via the Meta class
Saving data	form.save() writes directly to the database
Validation	Model-level constraints are also enforced
Use case	CRUD forms for model data entry/editing

Example: ModelForm vs Regular Form

Regular Form approach (repetitive):

```
# models.py
class Project(models.Model):
    topic    = models.CharField(maxlength=200)
    duration = models.IntegerField()

# forms.py — fields duplicated!
class ProjectForm(forms.Form):
    topic    = forms.CharField(max_length=200)
    duration = forms.IntegerField()
```

ModelForm approach (DRY):

```
# forms.py — no field duplication
class ProjectForm(forms.ModelForm):
    class Meta:
        model = Project   # All fields derived from model
```

Q11. Illustrating the Usage of Django Admin Interface *(10 Marks)*

The Django admin interface provides a complete, web-based data management tool. Its usage can be illustrated across four main areas:

1. The Login Screen

Visiting http://127.0.0.1:8000/admin/ presents a login screen. The superuser credentials (created during syncdb) are used to log in. The 'is_staff' and 'is_superuser' flags control access levels.

2. The Admin Index (Main Page)

After login, the main admin index displays all models with an Admin declaration, grouped by application. Links to 'Add' and 'Change' objects are shown for each model.

3. Change Lists

Clicking a model's 'Change' link opens the change list view — an index page of all objects of that type. With the Admin class configured:

```
class Admin:
    list_display  = ('title', 'publisher', 'publication_date')
    list_filter   = ('publisher', 'publication_date')
    ordering      = ('-publication_date',)
    search_fields = ('title',)
```

- list_display columns appear in the change list table.
- list_filter creates a sidebar for filtering by publisher, date range, etc.
- ordering sets the default sort order.
- search_fields creates a search box above the list.

4. Edit Forms

Clicking any object opens an edit form. Each field from the model appears with an appropriate widget. Django automatically uses:

- Calendar controls for date/time fields
- Select boxes for ForeignKey fields
- Text inputs for CharField fields
- Checkboxes for BooleanField fields

Edit forms validate input on submission. Errors are shown inline beside the offending fields. When an existing object is edited, a History button in the upper-right corner logs all changes.

5. Users, Groups, and Permissions

The admin interface includes User and Group management. Three key user flags:

- is_active: Whether the user is active at all.
- is_staff: Whether the user can log into the admin interface.
- is_superuser: Full unrestricted access to all admin items; ignores individual permissions.

Each editable model has three permissions: create, edit, and delete. These can be assigned to individual users or to groups for bulk assignment.

6. Deletion with Cascade Confirmation

Deleting an object shows a confirmation page that lists all related objects that will also be deleted. The user must confirm before the deletion proceeds, preventing accidental loss of data.

Q12. Django App: Student Course Registration with Admin Interface *(10 Marks)*

Below is a complete Django application that performs student course registration, registers admin interfaces, performs migrations, and supports data entry through admin forms.

Step 1 — Project & App Setup

```
django-admin.py startproject studentreg
cd studentreg
python manage.py startapp courses
```

Step 2 — models.py (Define Student and Course Models)

```
from django.db import models

SEMESTER_CHOICES = (
    ('1', 'First Semester'),
    ('2', 'Second Semester'),
    ('3', 'Third Semester'),
    ('4', 'Fourth Semester'),
)

class Course(models.Model):
```

```
    course_name = models.CharField(maxlength=100)
    course_code = models.CharField(maxlength=10)
    credits     = models.IntegerField()

    def __str__(self):
        return '%s (%s)' % (self.course_name, self.course_code)

    class Admin:
        list_display  = ('course_name', 'course_code', 'credits')
        search_fields = ('course_name', 'course_code')


class Student(models.Model):
    first_name   = models.CharField(maxlength=50)
    last_name    = models.CharField(maxlength=50)
    roll_number  = models.CharField(maxlength=20)
    email        = models.EmailField()
    semester     = models.CharField(maxlength=1, choices=SEMESTER_CHOICES)
    courses      = models.ManyToManyField(Course)

    def __str__(self):
        return '%s %s (%s)' % (self.first_name, self.last_name, self.roll_number)

    class Admin:
        list_display  = ('first_name', 'last_name', 'roll_number', 'semester')
        list_filter   = ('semester',)
        search_fields = ('first_name', 'last_name', 'roll_number')
        ordering      = ('last_name',)
```

Step 3 — settings.py (Configure Database and Installed Apps)

```
DATABASE_ENGINE = 'sqlite3'
DATABASE_NAME   = 'studentreg.db'

INSTALLED_APPS = (
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.auth',
    'django.contrib.admin',    # Enable admin
    'studentreg.courses',      # Our app
)
```

Step 4 — urls.py (Register Admin URLs)

```
from django.conf.urls.defaults import *

urlpatterns = patterns('',
    (r'^admin/', include('django.contrib.admin.urls')),
)
```

Step 5 — Perform Migrations (Create Tables)

```
python manage.py syncdb
# You will be prompted to create a superuser:
# Username: admin
# Email: admin@example.com
# Password: ******
```

Step 6 — forms.py (Registration Form)

```
from django import newforms as forms
from models import Student

class StudentRegistrationForm(forms.ModelForm):
    class Meta:
        model = Student
```

Step 7 — views.py (Registration View)

```
from django.http import HttpResponseRedirect
from django.shortcuts import render_to_response
from forms import StudentRegistrationForm

def register(request):
    if request.method == 'POST':
        form = StudentRegistrationForm(request.POST)
        if form.is_valid():
            form.save()   # Save student to database
            return HttpResponseRedirect('/register/success/')
    else:
        form = StudentRegistrationForm()
    return render_to_response('register.html', {'form': form})
```

Step 8 — register.html (Template)

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN">
<html lang="en"><head><title>Student Registration</title></head>
<body>
<h1>Student Course Registration</h1>
<form action="." method="POST">
    <table>{{ form.as_table }}</table>
    <p><input type="submit" value="Register"></p>
</form>
</body></html>
```

Step 9 — Running the Application

```
python manage.py runserver
# Admin interface:  http://127.0.0.1:8000/admin/
# Registration form: http://127.0.0.1:8000/register/
```

Data Entry Through Admin Interface

1. Log in at http://127.0.0.1:8000/admin/ using superuser credentials.
2. Click 'Add Course' to enter course records (name, code, credits).
3. Click 'Add Student' to register a new student with personal details, semester, and assigned courses.
4. Use the list_filter sidebar to filter students by semester.
5. Use the search_fields search box to find students by name or roll number.
6. Clicking the History button on any student record shows all admin changes made to that record.